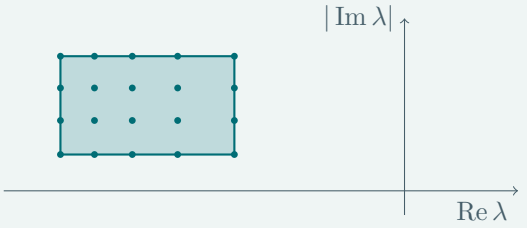
QUATERNIONIC SYSTEMS AND CONTROL

qstab

A public guide to quaternionic stability

Scalar polynomial families, right matrix spectra, and numerical evidence in MATLAB



Each dot represents one zero class.

The rectangle is the analytic image of $p(v) = v + a + b\mathrm{j}$, where a and $b > 0$ vary independently.

Read with MATLAB open All eleven numbered experiments have an executable key. They use real arrays or ordinary component structures and require no quaternion class. The same interfaces accept native MATLAB quaternion data.

Public release 0.18.0 / 9 November 2026

MATLAB R2024b Update 2: all 693 checks passed on candidate 0.18.0-rc.1. The numerical routines and examples are unchanged in this release.

Contents

1 Install, run and inspect 3

2 What data does the toolbox accept? 4

3 Understand the answer before choosing a solver 5

4 Forward shift, delay and state matrices 6

5 Choose the family that represents the experiment 7

6 Follow one ray, or search all directions 8

7 When the physical parameter has limits 9

8 Polytopes: sample the member before taking its norm 10

9 A cubic from the polytope study 11

10 From local evidence to a robust verdict 12

11 Coefficient balls and component boxes 13

12 What does a plot in two dimensions show? 14

13 Function directory 15

Example key In the Command Window enter `qstab_guide_example('G04')`, replacing `G04` with the question you want to explore.

Key	Experiment	Key	Experiment
G01	Algebraic zero vs. point value	G07	Six-generator cubic from the paper
G02	Delay reversal with grade	G08	Certificate and witness
G03	Matrix right eigenpairs	G09	Ball vs. component box
G04	First loss and later re-entry	G10	Explore a random interval
G05	Line vs. independent box	G11	Physical window and global radius
G06	Coarse and localized hulls		

Three routes For a first run read Sections 1–3. For a polytopic study begin with Sections 5, 8 and 9. For existing project data use the function directory at the end, then consult the named MATLAB help entry for options.

1 Install, run and inspect

Unpack the distribution into a writable folder. Open the directory containing `qstab_setup.m`; it is the package root. In a normal public MATLAB session:

```
qstab_setup('Examples',true);
qstab.about
catalogue = qstab_guide_example('list');
R = qstab_guide_example('G06');
R.ranges
```

The rows should be $[-3, 5]$, $[1, 5]$, $[1, 5]$. This one call exercises an actual value-set calculation and two levels of enclosure. To run all eleven short examples without opening figures, use `A=qstab_guide_example('all');`. Results are returned in memory; `A{6}.ranges` retrieves the same rows. These examples write no files. Most pages reproduce the body of an experiment directly from its MATLAB source. To see the whole file, use `type qstab_guide_example`. Section 9 includes its own generator construction in that source.

Which software is needed? Base MATLAB is sufficient for numeric arrays and ordinary component structures in these examples. The optional native input form uses the MathWorks quaternion class: use `which quaternion -all` to confirm which implementation your MATLAB finds. For example, Sensor Fusion and Tracking Toolbox provides this class; install a supported licensed product using MATLAB's Add-On Explorer. The active class should resolve under `matlabroot`. For large matrix Lyapunov solves the relevant Control System Toolbox functions may be useful. See `docs/installation.md` for product links and the optional external eigenvalue comparison. No external quaternion package, compiler, or download is needed for the guide. Avoid putting unrelated toolboxes on the search path to make these examples work.

Check the installation The package has a separate, fuller acceptance test. Run it in a fresh MATLAB session from the package root:

```
report = run_qstab_public;
report.archiveFile
assert(report.ok,'See the saved report: %s',report.archiveFile)
```

Unlike the examples, this command writes results: the named ZIP contains `report.txt` and `report.mat`. Copy an old report first if you want to retain it. The launcher temporarily restricts the search path and restores the caller's path and current directory. It does not edit the startup file. Native quaternion and graphics coverage require the corresponding MATLAB facilities. If a reader only wants the portable examples, `qstab_guide_example('all')` is the shorter entry point.

Optional external comparison For an independently installed right eigensolver, add only its entry directory to the session path, inspect `which qeig -all`, then pass its handle explicitly to `run_qstab_eigen_comparison` as `'LegacyFunction',@qeig`. This is separate from public acceptance. The author's qpe-public deals with evaluation zeros; it is not required here.

Where to look Type `help qstab.zeroexclude`, for instance. Qualified names distinguish the toolbox from MATLAB's ordinary `roots` and `eig`. The repository's `docs/reference` contains per-function pages; the README and citation file identify the software release and its companion work.

2 What data does the toolbox accept?

A quaternion $q = a + b\mathrm{i} + c\mathrm{j} + d\mathrm{k}$ has four real components. Its norm is $\sqrt{a^2 + b^2 + c^2 + d^2}$. The units obey $\mathrm{ij} = \mathrm{k}$, whereas $\mathrm{ji} = -\mathrm{k}$. The component labels specify directions in $\mathbb{H}$; they are not independent complex frequencies. A scalar polynomial has quaternion coefficients on the left and a central formal indeterminate v, so $vp_k = p_k v$ in polynomial arithmetic:

$$p(v) = p_0 + p_1 v + \cdots + p_g v^g, \qquad p_k \in \mathbb{H}.$$

The *grade* g is the shape declared for the model. The degree is the highest nonzero coefficient. An explicitly retained grade can exceed the degree.

Input	Meaning
`[2 3 1]`	Scalar $2 + 3v + v^2$. Powers increase from left to right; MATLAB's own polynomial commands usually expect the opposite order.
`quaternion(a,b,c,d)`	Four same-shaped real arrays, with positional order real, $\mathrm{i}, \mathrm{j}, \mathrm{k}$. Entries may represent coefficients, or a state matrix when selected explicitly.
`struct('a',a,...)`	Four same-shaped real arrays in fields `a,b,c,d`. Same values as the native form; no native constructor needed.
Coefficient pages	In a square matrix polynomial, page `P(:,:,k+1)` is the matrix multiplying v^k. For a scalar coefficient vector use a vector or $1 \times 1 \times (g+1)$ pages.
`{P1,P2,...}`	In a family call, polynomials defining a convex hull by default. In a single polynomial call, a cell of scalar coefficients can denote one polynomial. Identify the mathematical object first.

A common import mistake `qstab.polycomponents(P)` returns a real $4 \times (g+1)$ table with components in rows and powers in columns. The same unlabelled numeric matrix is *not* a polynomial input convention. To import it, use its rows as the four named fields. `qstab.components(A)` instead extracts four arrays matching the shape of the input, particularly useful for matrices.

No symbolic indeterminate is needed For $v - \mathrm{j}$, type the literal coefficient arrays in G01. If the native class is installed, the equivalent expression is `quaternion([0 1],[0 0],[-1 0],[0 0])`. Do not replace j and k by MATLAB's two ordinary complex spellings `i` and `j`: both of those ordinarily mean the same complex unit.

3 Understand the answer before choosing a solver

Write $p = a + b\mathrm{i} + c\mathrm{j} + d\mathrm{k}$, now with four *real polynomials* a, b, c, d. Its real norm polynomial is

$$N_p(v) = p(v)\overline{p}(v) = a(v)^2 + b(v)^2 + c(v)^2 + d(v)^2.$$

Here conjugation acts on the coefficients. These are ordinary polynomial products. At a complex point z, $a(z)^2$ is generally *not* $|a(z)|^2$. The useful feature is that one real polynomial records the algebraic zero classes of p. A nonreal quaternion $x + uy$ with $y > 0$ and $u^2 = -1$ belongs to a sphere of similar quaternions; all have the same real part and imaginary norm. The canonical representative in $\mathbb{C}$ is $x + \iota y$, with ι the complex unit. On a real class the sphere collapses to a point.

Executable example G01 `qstab_guide_example('G01')`

```
P = struct('a',[0 1],'b',[0 0],'c',[-1 0],'d',[0 0]); % s-j
[N,~] = qstab.normpoly(P);
[lambda,S] = qstab.roots(P);
assert(S.isValid && abs(lambda-1i)<1e-8)
assert(isequal(N(:).',[1 0 1]))
```

For $p(v) = v - \mathrm{j}$, $N_p = 1 + v^2$ has complex roots $\pm\iota$. The single class of the original degree-one polynomial is represented by $+\iota$. But right evaluation at quaternion i gives $\mathrm{i} - \mathrm{j} \neq 0$; evaluation at j gives zero. A representative identifies a class, not necessarily an orientation where direct substitution vanishes.

Deciding strict stability For a continuous scalar polynomial, strict stability requires every finite algebraic class to have negative real part, with the required degree retained. In the forward discrete variable it requires modulus less than one. `qstab.isstable(P)` normally tests a real norm polynomial by Routh or Schur recursion. Use `'Method','spectral'` to request an explicit spectrum as a separate check. The numerical test may decline to decide near the stability boundary.

`[tf,I]`	Interpretation for a stability query
`tf==1`	Strict stability was established to the method's floating-point acceptance criteria. For a robust query it refers to the whole specified family.
`tf==0`	Strict stability was refuted. Inspect the status and witness, including whether a boundary mode is confirmed.
`isnan(tf)`	No decision: budget, conditioning, contact ambiguity, or a failed assumption. Read `I.status` and `I.message` before changing tolerances.

Never use truthiness to interpret this three-way decision. The first output of a margin routine is instead a number or an interval; the first output of a spectral routine is a spectrum. Its detailed diagnostics follow.

4 Forward shift, delay and state matrices

Write $d = z^{-1}$. A delay coefficient vector is in increasing powers of d; it must not simply be relabelled as a forward vector. For fixed grade g the forward polynomial is

$$p_f(z) = z^g p_d(z^{-1}).$$

It reverses all $g+1$ coefficient slots, *including declared zeros*. A finite, nonzero delay root of modulus greater than one corresponds to a stable forward root inside the unit circle. A root at infinity in the delay representation becomes zero in the forward representation. A missing current-sample coefficient can cause a degree or regularity failure after reversal.

Executable example G02 `qstab_guide_example('G02')`

```
zP = [-0.5 1];   % z-1/2
dP = [1 -0.5];   % 1-d/2, d=z^-1
tfz = qstab.isstable(zP,'Variable','z');
tfd = qstab.isstable(dP,'Variable','zinv','Grade',2);
[delayRoots,I] = qstab.roots(dP,'Variable','zinv','Grade',2);
assert(tfz==1 && tfd==1 && I.isValid)
assert(any(isinf(delayRoots)) && any(abs(delayRoots-2)<1e-8))
```

Here $p_d = 1 - d/2$. At grade two the reversed polynomial is $z^2 - z/2$: its forward zeros are 0 and $1/2$. The delay list contains ∞ and 2. `qstab.roots(...,'Variable','zinv')` returns roots in the input variable; the decision checks the corresponding forward dynamics.

Matrices act from the left Right eigenpairs obey $Av = v\lambda$. Multiplying v on the right by a nonzero quaternion changes the reported eigenvalue to a similar quaternion. Hence a right spectrum can use the same class representatives as a scalar polynomial. Select `'Type','matrix'` when calling `qstab.isstable`, including for a 1×1 state matrix. Set `'Domain','discrete'` for a discrete-time state update.

Executable example G03 `qstab_guide_example('G03')`

```
A = struct('a',[-1 4;0 -2],'b',[1 0;0 0], ...
           'c',[0 1;0 1],'d',zeros(2));
[tf,I] = qstab.isstable(A,'Type','matrix');
[V,D,E] = qstab.eigpairs(A,'Output','components');
res = norm(qstab.chi(A)*qstab.chi(V)-qstab.chi(V)*qstab.chi(D),'fro');
assert(tf==1 && E.eigenpairsValid && res<1e-8)
```

The diagonal classes of this triangular 2×2 matrix are represented by $-1 + \iota$ and $-2 + \iota$. Its off-diagonal term affects directions. The complex adjoint χ sends an $m \times n$ quaternion matrix to a $2m \times 2n$ complex matrix and preserves matrix products. This is why the residual above checks the ordered equation $AV = VD$ without external quaternion matrix multiplication. Small residuals alone do not guarantee a basis: inspect `E.fullBasisObserved`. The nominal matrix test instead solves a Lyapunov or Stein equation when feasible.

5 Choose the family that represents the experiment

Kind	What may vary together?
`interval`	Each of the four real components of each coefficient independently satisfies a lower and upper bound. All componentwise orders must hold.
`polytope`	One common vector of nonnegative real weights summing to one mixes all coefficients of the generating polynomials. Generators need not all be extreme vertices.
`affine`	Real parameters in a box enter as $p_0 + \sum_r \theta_r g_r$. The same θ_r multiplies the whole generator polynomial.
`oneparameter`	One quaternion t is shared by all powers. Its side is part of the model: $p_0 + tp_1$ or $p_0 + p_1 t$. Bounds constrain real, i, j, k components of the same t.

Executable example G05 `qstab_guide_example('G05')`

```
L = [1 2 1]; U = [2 3 1];
F = qstab.family('interval',L,U);
line = qstab.family('affine',L,{U-L},[0 1]);
tfBox = qstab.robuststable(F);
tfLine = qstab.robuststable(line);
[K,~] = qstab.kharitonovcube({L,U});
tfCube = qstab.zeroexclude(K,'Uncertainty','affine');
assert(isequal([tfBox tfLine tfCube],[1 1 1]))
```

The box lets the constant and linear coefficient vary independently. The line varies both by exactly the same amount. For example $2 + 2v + v^2$ belongs to the box, but on the line a constant of 2 forces the linear coefficient to be 3. Both models happen to be Hurwitz: each real monic quadratic here has positive constant and linear coefficients. Matching decisions do not make the families equal.

Build once `qstab.family` returns ordinary MATLAB data; `qstab.randfamily` draws a well-defined example, not a proof that a random family is robustly stable. Specify `'Variable','z'` or `'Variable','zinv'` when appropriate, plus a declared grade if needed. The older explicit form `qstab.zeroexclude({L,U},'Uncertainty','interval')` remains useful. The same two entries without `'Uncertainty','interval'` ordinarily denote a segment, not a box.

A reduction with a precise scope For a continuous Cartesian quaternion coefficient interval, the Kharitonov construction uses eight shared real coordinates: four for even powers and four for odd powers. At each fixed imaginary-axis frequency its norm-value set agrees with the original interval's. The coordinates remain continuous: checking only 2^8 corners does not constitute the test. The even/odd independence argument does not automatically transfer to a discrete unit-circle boundary.

6 Follow one ray, or search all directions

For a nominally stable p_0, let $p_t = p_0 + tp_1$ and $t = \alpha u$, where u is a specified nonzero quaternion and α is *real*. Fixing u gives a one-dimensional experiment. The endpoints returned by `raymargin` bound the connected open interval of strict stability containing $\alpha = 0$. They are in units of α, not $|t|$. If u doubles, the endpoints halve. Continuous-time scalar calls require $\deg p_1 < \deg p_0$; the nominal leading term remains fixed.

Executable example G04 `qstab_guide_example('G04')`

```
P0 = [.5 1 1 1]; P1 = [4 1 1];
[ab,I] = qstab.raymargin(P0,P1,1,'Side','left');
alpha = [0 .5 2]; tf = zeros(size(alpha));
for n=1:numel(alpha)
    tf(n) = qstab.isstable(P0+alpha(n)*[P1 0],'Method','spectral');
end
assert(norm(ab-[-1/8,1-1/sqrt(2)],Inf)<1e-8)
assert(isequal(tf,[1 0 1]))
```

Here $p_\alpha = s^3 + (1+\alpha)s^2 + (1+\alpha)s + 0.5 + 4\alpha$. For a monic real cubic the nontrivial Routh inequality is

$$(1+\alpha)^2 > 0.5 + 4\alpha \quad \Longleftrightarrow \quad \alpha < 1 - 1/\sqrt{2} \ \text{ or } \ \alpha > 1 + 1/\sqrt{2}.$$

Combine this with positivity of the coefficients. Starting at zero, stability is first lost at $-1/8$ to the left, or $1 - 1/\sqrt{2} \approx 0.2929$ to the right. At $\alpha = 2$ the polynomial is stable again. The nominal branch does not describe all stable values on the line.



The teal intervals are computed above from the strict inequalities; the three black dots are the checks in G04.

State-matrix variant Specify `'Type','matrix'` for $A_0 + (tI)A_1$ or $A_0 + A_1(tI)$. The same scalar t does not generally commute with the entries of A_1. This is a fixed-ray result for a state matrix; it is not a robust test for arbitrary matrix polytopes. In discrete-time rays regularity events are tracked alongside boundary contacts. An unresolved endpoint is `NaN`; $\pm\infty$ and `NaN` mean different things.

7 When the physical parameter has limits

Suppose a calibration parameter is constrained to $B \subset \mathbb{H}$. This box defines the *admissible family*, useful for robust checks and sampling. The stable nominal law can still be deliberately studied beyond B. On a fixed line there are two separate questions:

$$J_{\mathrm{stab}} = (\alpha_-, \alpha_+), \qquad J_B = \{\alpha \in \mathbb{R} : \alpha u \in B\}.$$

A physical box edge is not evidence of dynamical contact. The part of the nominal stable branch that is physically allowed is $J_B \cap J_{\mathrm{stab}}$. Open and closed endpoints must be kept distinct. J_B can even be empty, a singleton, or a window not containing zero.

Executable example G11 `qstab_guide_example('G11')`

```
B = [-3 -1;0 0;0 0;0 0]; % real part in [-3,-1], imaginary parts fixed
F = qstab.family('oneparameter',[2 1],1,'Bounds',B,'Side','left');
[ab,I] = qstab.raymargin(F,-2);       % t=-2*alpha
admissible = I.parameterDomain.admissibleAlpha;
rho = qstab.freqmargin([2 1],1,[0 1]);
[bracket,J] = qstab.stabilityradius([2 1],1);
assert(ab(1)==-Inf && abs(ab(2)-1)<1e-8 && isequal(admissible,[.5 1.5]))
assert(norm(rho-[2 sqrt(5)],Inf)<1e-8)
assert(bracket(1)<=2+1e-9 && bracket(2)>=2-1e-9)
```

For $p_t = s + 2 + t$ and $u = -2$, the polynomial on the line is $s + 2 - 2\alpha$. It is strictly stable for $\alpha < 1$. The material constraint $-3 \le t \le -1$ independently gives $1/2 \le \alpha \le 3/2$. The physically admissible part of that stable branch is therefore $[1/2, 1)$; $1/2$ is a material edge and 1 a dynamical contact. The full-law nominal $t = 0$ is outside this particular box.

A different minimization `freqmargin` minimizes $|t|$ at a specified boundary frequency, over directions in quaternion space. At s-frequency ω in this example its answer is $\sqrt{4 + \omega^2}$. The global minimum over frequency is two, attained at $t = -2$, and `stabilityradius` returns a numerical bracket containing it. The unrestricted calls in G11 use the two polynomials explicitly. If called with a bounded family, requesting `'ExtendParameter',true` explicitly asks for the unrestricted law. It does not compute a radius restricted to B. For state matrices, use the documented ray interface; the all-direction scalar polynomial radius is a different algorithm.

8 Polytopes: sample the member before taking its norm

Let $p_w = \sum_{r=1}^{m} w_r p_r$ with real barycentric weights $w_r \geq 0$, $\sum w_r = 1$. At complex frequency ζ, take the four complex component values of each p_r as a vector $v_r \in \mathbb{C}^4$. The norm value of a member is

$$N_{p_w}(\zeta) = \sum_{r,s} w_r w_s B_{rs}, \qquad B_{rs} = v_r^{\mathsf{T}} v_s.$$

The transpose has *no* complex conjugation. A convex hull of these polarization values contains the actual set: $w_r w_s \geq 0$ sum to one. But the products are related, so most independent convex combinations of the B_{rs} need not be attainable. Even the convex hull of the generator *norm values* alone can miss actual member values.

Executable example G06 `qstab_guide_example('G06')`

```
Pm = struct('a',[1 1],'b',[0 0],'c',[-2 0],'d',[0 0]);
Pp = Pm; Pp.c(1)=2;
vertices = {Pm,Pp}; beta = linspace(0,1,41).';
[points,S] = qstab.valueset(vertices,0,'Weights',[1-beta beta]);
[coarse,C] = qstab.valueenclosure(vertices,0);
left=[1;0]; mid=[.5;.5]; right=[0;1];
[local,E] = qstab.valueenclosure(vertices,0, ...
                'VertexCells',{[left mid],[mid right]});
ranges = [coarse{1}.valueBounds(1,:); local{1}.valueBounds(1,:);
          local{2}.valueBounds(1,:)];
assert(isequal(ranges,[-3 5;1 5;1 5]))
assert(~C.zeroExcludedNumerically && E.zeroExcludedNumerically)
```

Use $p_t = s + 1 + t\mathrm{j}$, $-2 \leq t \leq 2$. At $s = 0$ the actual norm values are $1 + t^2$, so the exact range is $[1, 5]$. The mixed control between the ends is $1 - 4 = -3$. The global hull $[-3, 5]$ contains zero, even though every actual value is positive. Each half-segment has mixed control 1 instead, and either local hull excludes zero.



In `valueset`, `Weights` uses *one row per member*. In `valueenclosure`, each matrix in `VertexCells` uses *one column per local vertex*, expressed in the coordinates of the original generators. A user-specified subdivision must cover the family if its union is to describe all members. This calculation proves exclusion at one frequency only.

9 A cubic from the polytope study

Six quaternion polynomials illustrate why a false zero in an outer hull matters for a control model. Write $A = s + 1 + \mathrm{i}$, $B = s + 3 + \mathrm{k}$ and let u run through the four sign triples

$$(+,+,+), \quad (+,-,-), \quad (-,+,-), \quad (-,-,+).$$

For the first four generators set $C_u = s + 2 + 3(u_1\mathrm{i} + u_2\mathrm{j} + u_3\mathrm{k})$, $p_u = AC_uB$ *in that order*. Add $p_5 = p_1 + AB/10$ and $p_6 = p_1 + s^2/10$. The polynomial product commutes with the central s, not with the quaternion coefficients. The short helper at the bottom of the runnable file performs polynomial convolution with ordered Hamilton products. It needs only real arithmetic; it is not a general polynomial class.

Executable example G07 `qstab_guide_example('G07')`

```
vertices = cubicGenerators();  % six ordered quaternion polynomials
W = [eye(6); .25 .25 .25 .25 0 0];
[points,S] = qstab.valueset(vertices,0,'Weights',W);
[~,E] = qstab.valueenclosure(vertices,0);
C=zeros(4,6);
for n=1:6
    B=qstab.polycomponents(vertices{n}); C(:,n)=B(:,1);
end
G=C.'*C; hullZero=(5*G(1,1)+31*G(1,2))/36;
assert(abs(hullZero)<1e-9 && ~E.zeroExcludedNumerically)
assert(norm(points(:)-[620;620;620;620;628.2;620;80],Inf)<1e-8)
```

At $s = 0$ each $v_r \in \mathbb{R}^4$; the diagonal controls $v_r^{\mathsf{T}} v_r$ give the six generator values. The seventh weight row has equal weights on the first four. Their varying imaginary signs cancel *before* taking the norm: the seventh value is 80, although all first four generator values are 620. In fact the seventh constant is $A(0)\,2\,B(0)$, so its squared norm is $2 \cdot 4 \cdot 10 = 80$.

$$\underbrace{620, 620, 620, 620, 628.2, 620}_{\text{six generators}}, \qquad \underbrace{80}_{\text{one admissible mixture}}.$$

Meanwhile $B_{11} = 620$ and $B_{12} = -100$. The displayed independent convex combination $5B_{11}/36 + 31B_{12}/36 = 0$ puts zero in the *outer* hull. Those are weights on the controls, not weights on the original six generators. None of the seven sampled members supplies that zero. These concrete calculations explain the need for localization; they do not certify stability of this whole cubic family. The full argument is in M. Šebek, *A Sharp Face Theorem for Robust Stability of Quaternionic Polynomial Polytopes* (manuscript prepared for *Automatica*). The exact-data companion is separate from this finite MATLAB experiment: run `python reproducibility/qb03/replay.py` from the repository root to replay the mechanical and cubic rational checks. No Python installation is needed for the MATLAB guide. For package citations and publication updates see `docs/papers.md`.

10 From local evidence to a robust verdict

The zero-exclusion argument uses a connected parameter family, a strictly stable reference member, and a constant effective degree (or the corresponding forward regularity condition). Under these hypotheses a root cannot leave the open stability region without contacting its boundary. The numerical problem is to exclude the origin from the norm-value set *for every boundary frequency and every admissible parameter*. Finite samples at selected frequencies cannot discharge those quantifiers.

Choose a computational route `qstab.zeroexclude` subdivides a joint parameter–frequency search. For a polytope one may request `'Reduction','four-generator'`: the reduced tests concern whole convex hulls of up to four generators, including their interiors. This is not the classical recipe of checking only exposed edges, or a vertex stability test. It applies with the stated fixed-degree and reference requirements. The scalar `qstab.robuststable` also offers a Lyapunov subdivision route. Either numerical method can stop with unresolved regions.

Executable example G08 `qstab_guide_example('G08')`

```
good = {[1 1],[2 1]}; bad = {[-1 1],[-2 1]};
[tfGood,I] = qstab.zeroexclude(good,'StoreCertificates',true);
[rows,C] = qstab.certificateData(I);
[tfBad,J] = qstab.robuststable(bad);
[P,W] = qstab.witnessData(bad,J);
assert(tfGood==1 && ~isempty(rows))
assert(tfBad==0 && W.isAdmissible && ~isempty(P))
```

For the first family the real root lies between -2 and -1. For the second it lies between 1 and 2: one admissible member is enough to refute robust stability. `certificateData` reads the records already retained by the solver; `witnessData` reconstructs a polynomial from the original family and the saved report, checks admissibility and inspects its spectrum. Do not reuse a saved witness with different generators or different coefficient bounds.

How to diagnose an undecided case Read `I.status` and `I.message` first. If the family can lose a required coefficient, inspect `qstab.leadingbound`; a zero lower estimate alone does not establish a degree-loss witness. For a path such as $1+(1-2\theta)s$, the leading coefficient is $+1$ at one end and -1 at the other but zero at $\theta = 1/2$. Testing it only at the ends misses the problem. A degree loss is an assumption event, not a fabricated finite unstable root. If subdivision used its budget, retain the unresolved cells and increase `MaxTime`, `MaxCells` or `MaxDepth` only if the existing result warrants it. A larger budget can refine the answer; it does not change an assumption.

Inspect a cell Plot the retained cells and gaps with `plotCertificate`. To see the geometry, `certificateEnclosure` reconstructs a specific stored value-space cell; `plotEnclosure` displays the individual outer regions. A Lyapunov slack is not a norm-value gap. A plot of accepted frequency intervals cannot by itself establish coverage of the parameter domain.

11 Coefficient balls and component boxes

A *coefficient ball* allows a quaternion perturbation δ_k to every active coefficient. Given nonnegative real weights γ_k, its scaled size is

$$\left\| \left(|\delta_k| / \gamma_k \right)_{k:\gamma_k>0} \right\|_\eta , \qquad 1 \le \eta \le \infty .$$

A zero weight fixes that coefficient. The quaternion directions within each coefficient are isotropic. In a *component box*, the perturbations are bounded separately as $|\delta_{\nu k}| \le r W_{\nu k}$, where W is a real nonnegative $4 \times (g+1)$ array in the chosen coefficient basis. Now a zero width fixes just the selected component. Rotating the imaginary directions of the query can change what is attainable.

Executable example G09 `qstab_guide_example('G09')`

```
P = [2 1]; gamma=[1 0];
W = zeros(4,2); W(1,1)=1;    % only the real constant may change
[ball,B] = qstab.ballvalue(P,gamma,1i);
[box,C] = qstab.boxvalue(P,W,0);
assert(B.valid && C.valid && abs(ball-sqrt(5))<1e-8 && abs(box-2)<1e-8)
```

For $p = s+2$, allowing an arbitrary quaternion constant perturbation requires $|\delta_0| = |2+u| = \sqrt{5}$ to obtain a zero somewhere on the unit imaginary sphere. The ball query at the complex representative ι covers that *whole class*. The box here varies only the real constant; forcing a zero at the specific point zero requires $\delta_{00} = -2$, hence a scale of two. These figures illustrate two explicitly different questions; they need not agree.

Why the algorithms differ If $z = x + \iota y$ and $u^2 = -1$, the right evaluation $p_r(x+uy)$ can be written $e + ou$, where $e = \sum p_k \operatorname{Re}(z^k)$ and $o = \sum p_k \operatorname{Im}(z^k)$ are quaternions. For an isotropic weighted ball the distance to the class is determined by the least singular value of $\sum_k \chi(p_k) z^k$, a 2×2 complex matrix, divided by the appropriate dual norm of $(\gamma_k |z|^k)_k$ when sensitive. No fixed imaginary direction needs to be guessed. For component boxes, `boxvalue` solves a real feasibility/gauge problem for an *actual quaternion point*. Sampling a few points on a sphere does not prove a minimum over the class.

Question	Coefficient ball	Component box
One location	`ballvalue`, `ballset` (class membership)	`boxvalue` (point distance)
First global loss as scale increases	`ballradius` returns a bracket	`boxradius` returns a bracket
Specified closed scale robust?	`ballstable`	`boxstable`

A full-radius result must account for all boundary frequencies and possible loss of effective degree. If all admissible perturbations fix a component, do not replace its zero weight by a tiny positive number. Record the exponent, widths, grade, variable and termination status with a reported margin.

12 What does a plot in two dimensions show?

There are two different objects to display. A *zero plot* computes the algebraic zero classes of each sampled member. In the plane it places each class at $(\operatorname{Re}\lambda, |\operatorname{Im}\lambda|)$. Both coordinates and the norm of the representative are exact invariants of similarity. A three-dimensional drawing of one such class at constant real part uses an exact slice of its imaginary sphere, giving a circle. Its stability does not depend on which direction on that sphere is chosen.

A *value plot* fixes a frequency, constructs members, and then evaluates each one. Select the projection according to the question:

Projection	Coordinates; what a point at the origin means
`class`	$(\operatorname{Re} w, \|\operatorname{Im} w\|)$ for the actual evaluated quaternion w. It preserves $\|w\|$; the plotted origin means $w = 0$ at the sampled point. Imaginary directions are lost.
`slice`	$(\operatorname{Re} w, \langle \operatorname{Im} w, u\rangle)$ for a fixed pure unit u. The omitted length is in `offSlice`. A dot at the origin may mean a nonzero value perpendicular to the slice.
`normpoly`	$(\operatorname{Re} N_p(\zeta), \operatorname{Im} N_p(\zeta))$ at a central complex argument. Zero is an algebraic class contact. This is not the complex projection of the physical quaternion value.

Executable example G10 `qstab_guide_example('G10')`

```
F = qstab.randfamily(3,'interval','Seed',17);
[~,Z] = qstab.plotFamilyZeros(F,9,'Plot',false);
[~,V] = qstab.plotFamilyValues(F,1,9,'Projection','class','Plot',false);
[~,G] = qstab.plotFamilyValues(F,1,9,'Projection','normpoly','Plot',false);
dims = [size(V.sample.parameters); size(V.sample.weights)];
assert(isequal(dims,[9 12;9 0]) && isequal(size(Z.lambda),[3 9]))
assert(isequal(V.coefficients,G.coefficients))
```

For the seeded cubic the monic leading coefficient is fixed. Each of the other three powers has four free real parts: the result has nine rows and twelve parameter columns, with no convex weights. For a two-generator polytope one would instead find two common barycentric weights per row. Inspect `V.sample.parameterIndex` and `V.coefficients(:,:,n)` to identify an actual sampled member. For a drawing of the same data, request a plot explicitly:

```
R = qstab_guide_example('G10');
[ax,Z] = qstab.plotFamilyZeros(R.family,81);
[ax,V] = qstab.plotFamilyValues(R.family,1,81,'Projection','slice');
```

Use `'Sampling','grid'` for a deterministic design or `'Sampling','random','Seed',17` for repeatable draws. The count is the total number of sampled *members*, not the number per coordinate. With a physical input axis u, the substitution is $u\omega$ in continuous time, $\cos\theta + u\sin\theta$ for z and $\cos\theta - u\sin\theta$ for delay. Use radians per sample in discrete time. Sampled curves can be persuasive even when an unsampled member lies on the boundary.

13 Function directory

Use qualified names in MATLAB. Type `help qstab.raymargin` for its full syntax. A name by itself need not select the intended function. The tables show the mathematical return value first. Detailed argument checking, name/value options, tolerances and method-specific limits are in MATLAB help and the per-function reference pages.

Name after `qstab.`	Use and result
`about`	Release, numerical scope and dependencies.
`components`	Extract same-shaped real arrays a, b, c, d from quaternion, complex or real data.
`polycomponents`	Extract a real $4 \times (g+1)$ coefficient table; optional metadata distinguishes stored grade and actual degree.
`family`	Construct a reusable ordinary structure from an interval, polytope, affine law or shared quaternion law.
`randfamily`	Construct a random but well-defined family with a controlled degree, type and optional seed.
`familycomponents`	Inspect generator coefficients, weights or real parameter coordinates of a bounded scalar family.
`normpoly`	Real ascending coefficients of the central norm of one scalar polynomial.
`chi`	Complex adjoint of a quaternion matrix, applied coefficient by coefficient when appropriate.
`companion`	One output: normalized complex companion when possible. Two outputs: matrices of the generalized companion pencil, retaining the grade.
`roots`	Canonical complex representatives of algebraic zero classes, retaining multiplicity and infinite modes at a declared grade.
`eig`, `eigpairs`	Right class representatives; or (V, D) and numerical residual and basis information. Not a solver for left eigenvalues.
`isstable`	Strict nominal decision for scalar polynomials or square state matrices. Its nominal square polynomial-matrix branch is experimental and has additional regularity requirements.
`lyapoperator`	Real representation of $X \mapsto AX + XA^*$ on quaternion Hermitian matrices.
`robuststable`	Whole scalar-family decision using subdivision and Lyapunov or zero-exclusion criteria.
`zeroexclude`	Whole-family numerical zero exclusion with explicit variable, budget, optional reductions and retained coverage.
`leadingbound`	Lower estimate for the norm of the required coefficient over the entire family; read the separation or witness status.
`kharitonovcube`	Continuous-time interval reduction to an affine model with eight real continuous parameters.

Margins, evidence and displays

Name after `qstab.`	Use and result
`raymargin`	First signed endpoints along a fixed quaternion direction. A prescribed parameter window is reported separately.
`freqmargin`	Smallest quaternion parameter norm for a class contact at a prescribed frequency.
`stabilityradius`	Global all-direction bracket for one shared quaternion perturbing a scalar polynomial.
`ballvalue`, `ballset`	Pointwise class distance and membership in a spectral set for weighted coefficient balls. Membership is not a global stability decision.
`ballradius`, `ballstable`	First global loss bracket and strict decision at a chosen ball size.
`boxvalue`, `boxradius`, `boxstable`	Actual-point component-box gauge, global scale bracket, and strict decision for a given scale.
`valueset`	Sample complex *norm values* for chosen members and arguments. The result retains the shared weights or coordinates.
`valueenclosure`	Enclose norm values at one complex argument over a polytope or supported box/affine cell; keep the union of local regions.
`certificateData`, `certificateEnclosure`	Read stored coverage data; reconstruct one retained value-space enclosure, where supported.
`witnessData`	Reconstruct a particular polynomial and inspect admissibility and separate nominal diagnostics.
`plotSpectrum`	Display a supplied set of right eigenvalue or algebraic zero classes in 2D or an exact 3D slice.
`plotValueSet`, `plotEnclosure`	Display norm samples and separate outer regions. Show mixed controls without calling them actual family members.
`plotCertificate`, `plotWitness`	Display saved evidence: coverage records or a reconstructed member's classes.
`plotFamilyZeros`, `plotFamilyValues`	Sample actual members. Compute a cloud of algebraic zero representatives or display a class, slice or norm projection at one frequency.

Where the present release stops Family certification concerns scalar quaternion polynomials with the documented uncertainty models. Nominal matrix and selected fixed-ray matrix routines do not constitute a robust-polynomial-matrix solver. The code uses ordinary floating point with residual, scale and separation checks; a numerical pass is not a directed-rounding interval proof. A sampled zero cloud, an outer enclosure at one frequency, a verified unstable member and a positive global numerical certificate answer four different questions. State which one you obtained when reporting a result.